23CSE455_DP_Cas...
File

ROLE
You are a software engineer comparing how one design pattern is expressed in Java, Python, JavaScript and C++.

CONTEXT
- Source instance: Observer "Booking Lifecycle Notification" (attached C# files).
- Team format: a common_spec.md + one folder per language + measure.py (Factory Method versions attached as examples).

TASK
1. Write common_spec.md: participants, rules, error cases with exact messages, a shared fixture, and 8 test cases (order, duplicate subscribe, unsubscribe, a throwing observer is isolated, missing e-mail, and one per event kind).
2. Implement it in all four languages with a framework-free test runner each (PASS/FAIL per case, final "PASS n/8", non-zero exit on failure).
3. Adapt measure.py to compile, run once to save test_result.txt, then 1 warm-up + 5 timed runs (median), and print the snippet LOC.

CONSTRAINTS
- Same roles and names in every language; idiomatic interfaces; no third-party libraries.
- Keep it synchronous and state that simplification in the spec.
- Never report a result or a time you didn't get from a real run.

OUTPUT
All files, then the exact commands to run.

Worked for 3m 32s

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analysis errored

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Analyzed

Done — I used your uploaded case-study repository and prepared a standalone cross-language implementation of the **Observer pattern: Booking Lifecycle Notification**.

Complete project ZIP

Includes the shared specification, all four implementations, test runners, measurement script, test results, timing records, and run instructions.

Download Observer_Pattern_Cross_Language.zip ↗

Run instructions

Read the project README ↗

Read common_spec.md ↗

View the actual measurement output ↗

1. Verification results

I ran the measurement script in the current environment after compiling the Java and C++ implementations and executing all four test runners.

Java

71 implementation LOC

8/8 passed

Python
69 implementation LOC
8/8 passed

JavaScript
63 implementation LOC
8/8 passed

C++
102 implementation LOC
8/8 passed

All four test runners exited with code `0`. The results were obtained from actual runs, not estimated or fabricated.

2. Measured execution times

The script performed one initial test run to save `test_result.txt`, one discarded warm-up run, and five timed runs per language.

Language	Median process time
Java	158.466 ms
Python	45.511 ms
JavaScript	38.656 ms
C++	3.518 ms

These timings measure the **whole test-runner process**, including startup and output capture. They are not a direct comparison of the Observer algorithm's execution speed. The values are specific to the environment in which I ran the tests.

3. Exact commands to run

Extract the ZIP and open a terminal in the `Observer_Pattern` directory.

Run all four languages and regenerate results

```bash
python measure.py
```

This is the recommended command. It compiles where necessary, saves each language's test output, performs the timed runs, records the median, and prints implementation LOC.

Run each language individually

Java:

```bash
cd Java
javac BookingEventPublisher.java TestRunner.java
java TestRunner
```

Python:

```bash
cd Python
python -S test_runner.py
```

JavaScript:

```bash
cd JavaScript
node testRunner.js
```

C++ (requires a C++17 compiler):

```bash
cd Cpp
g++ -std=c++17 -O2 -Wall -Wextra -o test_runner.exe test_runner.cpp
./test_runner.exe
```

On Windows Command Prompt, use `test_runner.exe` instead of `./test_runner.exe`.

4. What is implemented

- `common_spec.md` defines the participants, shared fixture, synchronous simplification, exact warning message, and all eight test cases.
- Each language has its own framework-free test runner with `PASS` / `FAIL` per case, a final `PASS n/8` line, and a non-zero exit code if a test fails.
- The four ports preserve the Observer roles and the booking event behavior, including subscription order, duplicate-subscription prevention, unsubscription, and isolation of a throwing observer.
- Missing customer e-mail prevents e-mail creation while the audit observer still records the event.
- `measure.py` records actual test output in `test_result.txt` and raw timing measurements plus the median in `timings.txt`.

Before submission: keep `test_result.txt`, `timings.txt`, and `measure_output.txt` as generated evidence from the actual run. If you rerun the tests on your own machine, regenerate these files there because execution times will differ.

‹/›

g++ isn't installed, what's the easiest free way to compile the C++ one?

The easiest free option on your Windows laptop is **MSYS2 + MinGW-w64**. It installs a real C++ compiler locally, works with VS Code, and lets you run your C++ Observer tests without depending on an online compiler.

Option 1 — Install a compiler locally (recommended)

1. **Download MSYS2**

   Open the official website: msys2.org ↗ and install it using the default settings.

2. **Open the correct terminal**

   From the Start menu, launch **MSYS2 UCRT64**.

3. **Install GCC**

   Run this command in that terminal:

   ```
   bash     pacman -S --needed mingw-w64-ucrt-x86_64-gcc
   ```

Enter `Y` if asked to confirm.

4. **Add the compiler to Windows PATH**

   Add this folder to your Windows user `Path` environment variable:

   ```text
   C:\msys64\ucrt64\bin
   ```

   Restart VS Code or open a new terminal afterward.

5. **Verify the installation**

   In the VS Code terminal, run:

   ```powershell
   g++ --version
   ```

   If it prints the compiler version, you're ready.

Compile and run your C++ Observer tests

From your project's `Cpp` folder:

```powershell
g++ -std=c++17 -O2 -Wall -Wextra -o test_runner.exe test_runner.cpp
.\test_runner.exe
```

You should see eight individual test results and a final `PASS 8/8` if everything passes.

Option 2 — No installation

Use OnlineGDB ↗ if you only need to compile and test the C++ file once. Paste or upload the source code, select C++17, and click **Run**.

The downside is that your Python `measure.py` script cannot automatically use that website as a local `g++` compiler, so it won't be a drop-in replacement for the full measurement workflow.

My recommendation: install MSYS2. It's free, and once configured, you can compile and measure all four language implementations locally.

Observer_Pattern_...
File

From the attached real outputs, write: four rows for cross_language.csv (pattern_name, instance_name, language, snippet_path, source_loc, max_function_cc, runtime_compiler, test_command, test_result, median_process_ms, language_notes), counting CC as 1 + each if/loop/case/catch/&&/||; and a README per language with the version, the commands and the working folder. Copy every number exactly from the logs.
Write the Observer part of report section 4 under these headings: Selected instances, Common specifications, Implementations, Actual results, Static and dynamic comparison, Comparison (structure, abstraction, inheritance/composition, identity, ownership, error handling), Technology discussion (3-4 sentences, real sources only: the GoF book, Microsoft Learn, Node.js docs). Use only the attached files and logs.

Worked for 1m 1s

I've prepared the Observer cross-language deliverables from your attached ZIP, using the actual `measure_output.txt`, individual `timings.txt` files, `test_result.txt` files, and implementation sources as evidence.

1. Download the completed files

Complete deliverables ZIP

Includes the original project evidence, updated README files for all four languages, `cross_language.csv`, and the Observer report section.

Download Observer_Pattern_Deliverables.zip ↗

Cross-language CSV

Exactly four data rows with the requested 11 columns.

Download cross_language.csv ↗

Report Section 4

Contains all seven requested headings, including the static and dynamic comparison, language-specific trade-offs, and technology discussion.

Read section4_observer.md ↗

2. Per-language README files

Each README records the tool version from the logs, the working folder, exact compile/run commands, measured snippet LOC, actual test result, median process time, and language-specific implementation details.

Java
`javac 21.0.12.1` · 71 LOC — README.md ↗

Python
`Python 3.13.5` · 69 LOC — README.md ↗

JavaScript
`v22.16.0` · 63 LOC — README.md ↗

C++
`g++ (Debian 14.2.0-19) 14.2.0` · 102 LOC — README.md ↗

3. Actual results copied from the logs

Language	Test result	Median process time
Java	`PASS 8/8` , exit 0	158.466 ms
Python	`PASS 8/8` , exit 0	45.511 ms
JavaScript	`PASS 8/8` , exit 0	38.656 ms
C++	`PASS 8/8` , exit 0	3.518 ms

The CSV preserves the exact recorded version strings, snippet LOC, commands, and medians. The maximum function CC values are calculated separately using your specified rule: baseline 1 plus each `if` , loop, `case` , `catch` , `&&` , or `||` .

Language	Maximum function CC
Java	9
Python	3

JavaScript	3
C++	6

These CC values are manual counts from the supplied implementation source, not measurements reported by the timing logs. The report explains the counting method.

4. Technology references used in the report

The discussion uses the Gang of Four book, *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma et al., Addison-Wesley, 1994), and these official documentation sources:

- Microsoft Learn — Observer design pattern ↗ Microsoft Learn
- Microsoft Learn — Observer design pattern best practices ↗ Microsoft Learn
- Node.js — Events documentation ↗ GitHub

One important distinction is preserved throughout: the four implementations are **synchronous, in-memory comparison ports**. Their process timings include runtime startup and output capture, so they should not be presented as direct measurements of Observer notification performance in isolation.